

Check If a String Contains All Binary Codes of Size K

Authored by: Arpit Raj • LNMIIT Jaipur • July 1, 2026

Given a binary string s and an integer k , return `true` if every binary code of length k is a substring of s . Otherwise, return `false`.

Example Walkthrough:

Input: $s = "00110110"$, $k = 2$

Total possible binary strings of size $k = 2$ is $2^k = 2^2 = 4$ codes: $\{ "00", "01", "10", "11" \}$.

Scanning substrings of size 2:

- `"00"` → Found
- `"01"` → Found
- `"10"` → Found
- `"11"` → Found

Since all 2^k codes are present, the result is `true`.

Approach 1: Brute Force (Sliding Window via String Set)

The intuitive approach uses a fixed-size sliding window of length k . We slide across the string from left to right, capturing every substring of length k . These substrings are inserted into an `unordered_set<string>` to automatically handle duplicates.

The process begins by extracting the first window at index 0. For each subsequent step, the window is updated efficiently by erasing the oldest character from the front and appending the new incoming character to the back. At the end, if the set's size equals 2^k , it confirms that all possible binary codes are present.

```

class Solution {
public:
    bool hasAllCodes(string s, int k) {
        int n=s.size();
        unordered_set<string> st;

        string sus=s.substr(0,k); //pehla window
        st.insert(sus);

        for(int j=k;j<n;j++){
            sus.erase(sus.begin());
            sus.push_back(s[j]);
            st.insert(sus);
        }

        if(st.size()==pow(2,k)) return true;
        else return false;
    }
};

```

Complexity Analysis:

Time Complexity	$O(n \times k)$ — Extracting, copying, or hashing a substring of length k costs $O(k)$ time, performed for approximately n windows.
Space Complexity	$O(2^k \times k)$ — There are at most 2^k unique binary codes stored in the set, where each code string consumes k bytes of memory.

Approach 2: Rolling Hash & Bit Manipulation (Optimized)

To optimize both time and space, we avoid expensive string allocations, hashing, and deep-copy operations by treating each binary substring as its integer decimal representation. This transforms our storage layout into an `unordered_set<int>`, reducing hashing overhead to $O(1)$.

Binary-to-Decimal Mapping Representation (for $k = 3$)

Binary Substring	Integer Representation
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Sliding Window Bitwise Operations

As the window slides, the integer value (or bit-mask) is dynamically adjusted in **$O(1)$** time using two primary steps:

- **Add Incoming Bit:** Shift the current mask to the left by 1 position ($\text{mask} \ll= 1$) to clear the leftmost spot, then apply a bitwise OR with the new digit value ($\text{mask} |= (\text{s}[\text{j}] - '0')$).
- **Remove Outgoing Bit:** Since the window must track exactly k bits, any bit shifted beyond index $k-1$ must be removed. This is achieved using an AND-mask filter ($\text{remover} = (1 \ll k) - 1$).

Bitwise Custom Execution Example

Let $k = 3$. The bitwise remover mask is computed as: $(1 \ll 3) - 1 = 8 - 1 = 7$, which yields 0111 in binary (retaining exactly 3 bits).

```
1011 (Current bit-state after insertion and shift)
& 0111 (Remover mask for k=3)
-----
0011 (Resulting window: Leftmost extra '1' cleanly cut off!)
```

```

class Solution {
public:
    bool hasAllCodes(string s, int k) {

        int n=s.size();
        if(k>n) return false;
        unordered_set<int> st;

        int mask=0;
        int remover=(1<<k) -1;
        for(int i=0;i<k;i++){
            mask<<=1;
            mask|=s[i]-'0';

        }
        st.insert(mask);

        for(int j=k;j<n;j++){
            mask<<=1;
            mask|=s[j]-'0';
            mask&=remover;

            st.insert(mask);

        }

        if(st.size()==pow(2,k)) return true;
        return false;
    }
};

```

Complexity Analysis:

Time Complexity	$O(n)$ — Hashing integers and running primitive bitwise operations takes deterministic $O(1)$ time per window over the string string size n .
Space Complexity	$O(2^k)$ — The hash set stores up to 2^k distinct integer mappings, each occupying constant primitive space.